PERSONAL LEARNING ROADMAP

Full-Stack AI Engineer 6-Month Roadmap

A complete, project-driven path from CSS fundamentals to a production-grade full-stack application — built to combine your existing AI / ML / DL / NLP / GenAI / Agents / RAG / MLOps skills with modern frontend and backend engineering.

22–24	12	11+	1
WEEKS	PHASES	PROJECTS	CAPSTONE

HTML → CSS → Tailwind CSS → JavaScript → Git/GitHub → React → TypeScript → Next.js → Python (FastAPI) → SQL + NoSQL → Auth & Security → Docker & Deployment → Capstone (Full-Stack + AI)

How To Use This Roadmap

Read this once before you start. It explains the structure and the learning rule that ties every phase together.

> **The core rule:** Never move to the next phase until you've built the project attached to the current one. Concepts you only "read about" disappear in 2 weeks. Concepts you build with stay forever. Every phase below ends with a project, and every project deliberately reuses what you built in the phase before it — so by the end you're not learning 12 separate things, you're building one growing application.

Daily rhythm (suggested)

- ☐ 1.5–2 hrs on weekdays, 3–4 hrs on one weekend day
- ☐ 70% building / coding, 30% reading docs or watching a video
- ☐ Push every project to GitHub — this becomes your portfolio
- ☐ Keep a short daily log: what you built, what broke, what you fixed

Since you already know AI/ML

- ☐ This roadmap does **not** re-teach AI/ML/DL/NLP/GenAI/RAG/Agents/MLOps
- ☐ Phase 12 (Capstone) is where your AI skills plug into the stack you're about to build
- ☐ Python basics are skipped — you go straight into FastAPI in Phase 9
- ☐ Think of this as: "AI engineer who can now ship the whole product," not just the model

The 12 phases at a glance

#	Phase	Duration	Focus	Project
1	HTML5 + CSS3	Week 1–2	Structure & styling fundamentals	Multi-page personal portfolio site
2	Tailwind CSS	Week 3	Utility-first styling, responsive design	Rebuild portfolio in Tailwind + landing page
3	JavaScript (ES6+)	Week 4–6	Logic, DOM, async, APIs	Weather app / expense tracker (API-driven)
4	Git, GitHub & Dev Tools	Week 7	Version control, deployment basics	Deploy all previous projects live
5	React	Week 8–10	Components, hooks, state, routing	Full CRUD app (e.g. task/notes manager) with a mock API
6	TypeScript	Week 11	Type safety for JS & React	Migrate React project to TypeScript
7	Next.js	Week 12–13	SSR/SSG, routing, API routes, deployment	Full-stack blog / SaaS landing app
8	Frontend Capstone Checkpoint	Week 14	Polish + combine everything so far	Production-ready frontend app, deployed
9	Python + FastAPI	Week 15–16	REST APIs, async, validation, docs	REST API for the app you built in React/Next
10	Databases (SQL + NoSQL)	Week 17–18	PostgreSQL, SQLAlchemy, MongoDB, Redis	Persist the FastAPI app to a real database
11	Auth, Security, DevOps	Week 19–21	JWT/OAuth, testing, Docker, CI/CD	Add login system + containerize + deploy
12	Capstone: Full-Stack AI Engineer	Week 22–24	Everything + your existing AI/GenAI/RAG skills	One complete, deployed full-stack AI product

Frontend Track

Phases 1–8. Goal: by the end of this track you can independently design, build, and deploy a polished, responsive, type-safe web application.

Phase 1 — HTML5 & CSS3 Fundamentals

Week 1–2

> You asked to start at CSS — but 2–3 days of HTML structure first will make everything else click faster. It's short, don't skip it.

HTML5

- Semantic tags (header, nav, main, section, article, footer)
- Forms & input types, validation attributes
- Tables, lists, media (img, video, audio)
- Accessibility basics (alt text, labels, ARIA roles)
- SEO basics: meta tags, Open Graph

CSS3 — Core

- Selectors, specificity, the cascade, inheritance
- Box model, margin/padding/border, box-sizing
- Typography, colors, units (px, %, rem, em, vh/vw)
- Positioning (static, relative, absolute, fixed, sticky)
- Display: block, inline, inline-block, none

CSS3 — Layout & Beyond

- Flexbox (full: main/cross axis, wrap, gap, align/justify)
- CSS Grid (template columns/rows, areas, auto-fit/fill)
- Responsive design & media queries, mobile-first
- Transitions, transforms, keyframe animations
- CSS variables, pseudo-classes/elements, z-index/stacking

Project 1 — Personal Portfolio Site (multi-page, plain HTML/CSS)

Home, About, Projects, Contact pages. Fully responsive (mobile/tablet/desktop) using Flexbox + Grid, one CSS animation, a working contact form (frontend only). No frameworks — this proves you understand raw CSS before you start using shortcuts.

Reference docs: developer.mozilla.org (MDN) for HTML & CSS, web.dev/learn, CSS-Tricks (Flexbox & Grid guides)

Phase 2 — Tailwind CSS

Week 3

Core Concepts

- Utility-first workflow, why Tailwind vs raw CSS
- Setup (CLI / Vite / PostCSS), tailwind.config.js
- Spacing, sizing, color, typography utilities
- Flexbox & Grid utilities

Responsive & State

- Responsive breakpoints (sm, md, lg, xl, 2xl)
- State variants: hover, focus, active, disabled
- Dark mode (class strategy)
- Group & peer modifiers

Scaling Up

- @apply and custom components
- Design tokens / theme customization
- Component libraries built on Tailwind (shadcn/ui, DaisyUI) — awareness level
- Performance: purge/content config

Project 2 — Rebuild + Landing Page

Rebuild your Phase 1 portfolio entirely in Tailwind (compare code volume & speed). Then build a SaaS-style landing page (hero, features grid, pricing table, FAQ accordion, footer) with dark mode toggle — this becomes a reusable template for your later projects.

Reference docs: tailwindcss.com/docs

Phase 3 — JavaScript (ES6+)

Week 4–6

LANGUAGE FUNDAMENTALS

- Variables (let/const), data types, type coercion
- Operators, control flow, loops
- Functions, arrow functions, scope, closures
- Arrays & array methods (map, filter, reduce, find, sort)
- Objects, destructuring, spread/rest operators
- Template literals, optional chaining, nullish coalescing

DOM & BROWSER

- DOM selection & manipulation, event handling
- Event bubbling/capturing, delegation
- Forms & validation in vanilla JS
- LocalStorage / SessionStorage
- Browser dev tools & debugging

MODERN JS & ASYNC

- ES6 modules (import/export)
- Classes, prototypes, `this` keyword
- Promises, async/await, error handling (try/catch)
- Fetch API, working with REST JSON APIs
- JSON parsing/stringifying, debouncing/throttling

Project 3a — Interactive UI Set

Build 3 small vanilla-JS apps: a to-do list (localStorage persistence), a quiz app, and a countdown/timer app. Focus: DOM manipulation and state without a framework.

Project 3b — API-driven App (combines JS + Tailwind)

A weather dashboard or expense tracker that fetches data from a real public API, handles loading/error states, and is styled entirely with Tailwind. This is your first "real" full JS + CSS project.

Reference docs: javascript.info, MDN JavaScript Guide, freeCodeCamp JS curriculum

Phase 4 — Git, GitHub & Developer Tooling

Week 7

GIT

- init, add, commit, status, log, diff
- Branching, merging, rebasing basics
- .gitignore, resolving merge conflicts
- Undoing changes (reset, revert, checkout)

GITHUB

- Remote repos, push/pull, forks, pull requests
- README writing, GitHub Pages (static hosting)
- Issues & project boards (basic workflow)

TOOLING

- npm/pnpm basics, package.json, semantic versioning
- VS Code shortcuts & extensions
- Chrome DevTools (Network, Console, Elements)
- Intro to deployment (Vercel/Netlify for static/React sites)

Project 4 — Portfolio Goes Live

Push everything built so far to GitHub with clean commit history and READMEs. Deploy the portfolio and the API-driven app live (Vercel/Netlify) with custom domains if possible. This is non-negotiable — an undeployed project isn't a portfolio piece yet.

Phase 5 — React

Week 8–10

CORE

- JSX, components, props, composition
- useState, useEffect (dependency arrays, cleanup)
- Conditional rendering, lists & keys
- Forms & controlled inputs
- Component lifecycle mental model

INTERMEDIATE

- useRef, useMemo, useCallback, useContext
- Custom hooks
- Lifting state up, prop drilling & when to avoid it
- React Router (routes, dynamic params, nested routes)
- Fetching data (loading/error/empty states)

STATE MANAGEMENT & ECOSYSTEM

- Context API for global state
- Zustand or Redux Toolkit (pick one, learn well)
- React Query / TanStack Query (server state caching)
- Form libraries: React Hook Form + Zod validation
- Component testing basics (React Testing Library)

Project 5 — Full CRUD App (combines React + Tailwind + JS)

A notes/task manager or recipe app with create/read/update/delete, search & filter, routing between list/detail pages, and a mock backend (JSON Server or a public API). Styled with Tailwind, deployed live.

Reference docs: react.dev (official docs, use these — not old tutorials), tanstack.com/query

Phase 6 — TypeScript

Week 11

FUNDAMENTALS

- Basic types, type inference, type annotations
- Interfaces vs types, union & intersection types
- Arrays, tuples, enums
- Functions: typed params/returns, optional/default params

INTERMEDIATE

- Generics (functions, interfaces, components)
- Type narrowing, type guards
- Utility types (Partial, Pick, Omit, Record)
- tsconfig.json essentials, strict mode

WITH REACT

- Typing props, state, and events
- Typing hooks (useState<T>, custom hooks)
- Typing API responses (and validating with Zod)

Project 6 — Migrate to TypeScript

Convert your Phase 5 CRUD app from JS to TS: type every component prop, API response, and form. This is one of the highest-leverage exercises in the whole roadmap for job-readiness — almost all React jobs today require TS.

Reference docs: typescriptlang.org/docs

Phase 7 — Next.js

Week 12–13

CORE (APP ROUTER)

- File-based routing, layouts, nested routes
- Server Components vs Client Components
- Data fetching, caching & revalidation
- Dynamic routes, route groups, loading/error UI

RENDERING & APIS

- SSR, SSG, ISR — when to use each
- API routes / Route Handlers (backend-in-frontend)
- Middleware, environment variables
- Image/font optimization, metadata & SEO

PRODUCTION CONCERNS

- Auth integration (NextAuth/Auth.js)
- Connecting to an external API (your future FastAPI backend)
- Deployment on Vercel, environment config per stage

Project 7 — Full-Stack Next.js App

A blog/SaaS-style app with public pages (SSG), a dashboard (client-rendered), and its own API routes for CRUD — still using a mock/embedded database (SQLite or a hosted free Postgres) so it's genuinely full-stack end to end.

Reference docs: nextjs.org/docs

Phase 8 — Frontend Capstone Checkpoint

Week 14

One consolidation week before you switch to backend. Goal: everything from Phases 1–7 living in one clean, deployed, professional app.

Frontend Milestone Project

Take your Project 7 app and polish it to portfolio quality: full responsive design, dark mode, loading/error/empty states everywhere, accessibility pass, TypeScript throughout, clean component architecture, and a real deployed URL + README with screenshots. This is the app you link on your resume for "frontend."

Backend & Data Track

Phases 9–11. Since you already know Python, this moves fast into FastAPI, databases, and production concerns.

Phase 9 — Python for Web + FastAPI

Week 15–16

PYTHON WEB REFRESHER

- Virtual envs (venv/poetry/uv), project structure
- Type hints, dataclasses (used heavily by FastAPI)
- Decorators & context managers (used everywhere in FastAPI)
- HTTP fundamentals: methods, status codes, headers, REST principles

FASTAPI CORE

- Path/query/body params, Pydantic models & validation
- Path operations, routers, dependency injection
- Request/response models, status codes, error handling
- Auto docs (Swagger/OpenAPI), CORS setup
- Async endpoints (async def), background tasks

BEYOND BASICS

- File uploads, pagination, filtering, sorting
- WebSockets (for real-time features)
- Middleware, custom exception handlers
- Project structure for larger apps (routers/ services/schemas pattern)

Project 8 — REST API for Your Frontend

Build a FastAPI backend that replaces the mock data in your Phase 7/8 frontend app (same domain — notes, tasks, blog, whatever you chose). In-memory or SQLite storage first — the goal here is API design, not persistence yet.

Reference docs: fastapi.tiangolo.com

Phase 10 — Databases (SQL + NoSQL)

Week 17–18

RELATIONAL (POSTGRESQL)

- Tables, keys, relationships, normalization basics
- SQL: SELECT/JOIN/GROUP BY/ subqueries/indexes
- Transactions & ACID basics

ORM & MIGRATIONS

- SQLAlchemy (models, sessions, relationships) or SQLModel
- Alembic migrations
- Connecting FastAPI ↔ PostgreSQL properly (dependency-injected sessions)

NOSQL & CACHING

- MongoDB basics (documents, collections) — when to choose it over SQL
- Redis: caching, sessions, rate limiting
- Vector databases at a conceptual level (you likely already know this from RAG — note where it plugs into this stack)

Project 9 — Persist Everything

Migrate Project 8's API from in-memory/SQLite to a real PostgreSQL database via SQLAlchemy + Alembic migrations. Add Redis caching on your most-read endpoint. Your frontend (Phase 8 app) should now be talking to a real, persistent, multi-user backend.

Reference docs: postgresql.org/docs, docs.sqlalchemy.org, redis.io/docs, mongodb.com/docs

Phase 11 — Auth, Security, Testing & DevOps

Week 19–21

AUTH & SECURITY

- Password hashing (bcrypt/argon2), JWT access & refresh tokens
- OAuth2 flow, "Sign in with Google/GitHub"
- Role-based access control (RBAC)
- CORS, rate limiting, input sanitization, secrets management (.env)

TESTING

- pytest for FastAPI (unit + integration tests)
- Jest / React Testing Library for frontend components
- Basic end-to-end testing awareness (Playwright/Cypress)

DEVOPS & DEPLOYMENT

- Docker: Dockerfile, docker-compose (frontend + backend + db)
- Environment configs per stage (dev/staging/prod)
- CI/CD basics with GitHub Actions (lint → test → deploy)
- Deploy backend (Render/Railway/Fly.io/AWS) + frontend (Vercel) + managed Postgres
- Logging & basic monitoring/error tracking (e.g. Sentry) — awareness level

Project 10 — Ship It Properly

Add full JWT-based auth (signup/login/protected routes) to Project 9, write tests for your critical endpoints and components, Dockerize the whole stack with docker-compose, set up a GitHub Actions pipeline, and deploy the final version live with a custom domain.

The Capstone

Phase 12 — Week 22–24. This is where you stop being "someone who learned full-stack" and become a Full-Stack AI Engineer.

Capstone Project — Full-Stack AI Product

Design and ship one complete product that uses everything above **plus** the AI/ML/GenAI/RAG/Agent/LLM skills you already have. Pick one direction and go deep rather than shallow across many.

- **Idea A — AI SaaS:** A Next.js + TypeScript frontend, FastAPI backend, Postgres + vector DB, JWT auth, subscription-style dashboard, that wraps an LLM/RAG pipeline you already know how to build (e.g. "chat with your documents," an AI research assistant, or a domain-specific copilot).
- **Idea B — Agentic App:** A multi-step AI agent (using your existing agent/LLM knowledge) exposed through a FastAPI backend with streaming responses (WebSockets/SSE) and a polished React/Next.js UI showing the agent's reasoning/steps in real time.
- **Idea C — MLOps-integrated App:** A full-stack app where users interact with one of your trained/fine-tuned models via a FastAPI inference endpoint, with a proper CI/CD + monitoring setup connecting your MLOps skills to real users.

Required for all three: authenticated multi-user app, persistent database, Dockerized, tested, deployed live, documented on GitHub with a clean README, demo video/GIF, and architecture diagram.

Capstone checklist

- ☐ Responsive, accessible, TypeScript React/Next.js frontend
- ☐ FastAPI backend with proper routing, validation, and error handling
- ☐ PostgreSQL (+ vector DB / Redis where relevant) with migrations
- ☐ JWT or OAuth authentication with protected routes on both frontend and backend
- ☐ Your AI/GenAI/RAG/Agent component integrated as a real feature, not a demo script
- ☐ Tests on critical paths (backend + frontend)
- ☐ Dockerized with docker-compose, CI/CD via GitHub Actions
- ☐ Deployed live with a public URL, custom domain if possible
- ☐ Polished README: problem, architecture diagram, tech stack, screenshots, live link

After this roadmap: you'll have 4–5 deployed, documented, portfolio-quality projects (portfolio site, CRUD app, Next.js app, FastAPI+DB app, and the capstone) plus GitHub history showing consistent progress. That combination — real shipped products + AI specialization — is exactly what "Full-Stack AI Engineer" job postings ask for. Keep iterating on the capstone after this roadmap ends; it's your strongest interview talking point.